Student Budget Tracker

Midterm Project — Step-by-Step Instructions

IT 431 — Advanced Web Development

George Mason University

Spring 2026

200 Points

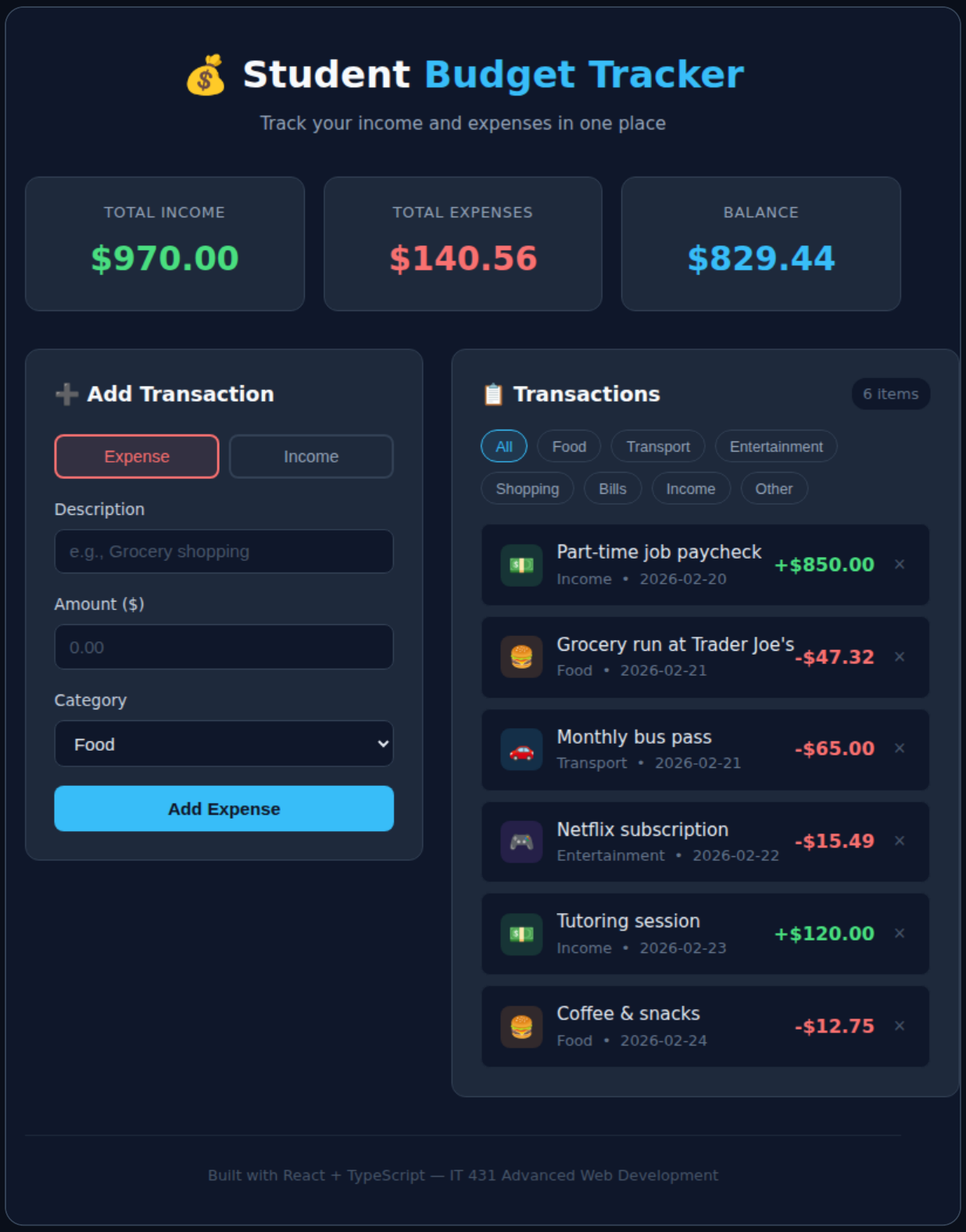
📋 Project Overview

In this project you will build a **Student Budget Tracker** — a React + TypeScript web application that lets users track their income and expenses. You will use everything we have covered so far: React components, props, state management with `useState`, event handling, TypeScript interfaces, and list rendering.

🎯 Learning Objectives

- Build a multi-component React application from scratch
- Define and use TypeScript interfaces for type-safe props
- Manage application state with the `useState` hook
- Handle user events (form submission, button clicks)
- Render dynamic lists using `.map()` with keys
- Implement conditional rendering
- Deploy a React app to GitHub and Vercel

What the Finished App Looks Like



Live demo: https://student-budget-tracker-smoky.vercel.app

What's Provided For You

File	What It Contains	Edit?
`App.css`	All CSS styling for the entire app	No
`types.ts`	TypeScript interfaces (`Transaction` , `Category` , `BudgetSummary`)	No
`main.tsx`	React entry point	No
`App.tsx`	Main app component (skeleton with TODOs)	**Yes**
`components/*.tsx`	Four component skeletons with TODOs	**Yes**

🛠️ Part 1: Project Setup

The starter project is ready to go — just clone, install, and start coding.

1 Clone the Starter Repository

```
git clone https://github.com/rmichak/it431-budget-tracker-project-starter.git
cd it431-budget-tracker-project-starter
```

2 Install Dependencies

```
npm install
```

3 Start the Dev Server

```
npm run dev
```

Open the URL shown in your terminal (usually `http://localhost:5173`). You'll see the app header and footer but the middle will be mostly empty — that's expected!

> 💡 **Keep the Dev Server Running**
>
> Leave `npm run dev` running while you work. Vite automatically refreshes the browser every time you save a file.

📐 Part 2: Understand the TypeScript Types

Before you start coding, read through `types.ts`. These are the data shapes your entire app is built around. **You do not need to edit this file.**

`src/types.ts`

```
export interface Transaction {
  id: number;
  description: string;
  amount: number;
  category: Category;
  type: "income" | "expense";
  date: string;
}

export type Category =
  | "Food" | "Transport" | "Entertainment"
  | "Shopping" | "Bills" | "Income" | "Other";

export interface BudgetSummary {
  totalIncome: number;
  totalExpenses: number;
  balance: number;
}
```

💡 Key Takeaways

- Every transaction has an `id`, `description`, `amount`, `category`, `type` ("income" or "expense"), and `date`
- `Category` is a union type — it can only be one of those specific strings
- `BudgetSummary` holds the calculated totals that the summary cards display

🧩 Part 3: TransactionItem Component

Start with the smallest component and work your way up.

`src/components/TransactionItem.tsx`

This component displays a single transaction row — the category icon, description, date, amount, and a delete button.

TODO 1 Define the Props Interface

📖 What This Means

Every React component that receives data from its parent needs a **props interface**. This tells TypeScript exactly what data the component expects. Think of it as a contract — "I need a `transaction` object and a `onDelete` function to work properly."

🔴 You'll See This (starter code)

```
// TODO 1: Define a TypeScript interface called "TransactionItemProps" with:
//         - transaction: Transaction
//         - onDelete: a function that takes (id: number) and returns void
```

⬇️

🟢 **Change It To This**

```
interface TransactionItemProps {
  transaction: Transaction;
  onDelete: (id: number) => void;
}
```

Why: `transaction: Transaction` means this component will receive one transaction to display. `onDelete: (id: number) => void` is a callback function — when the user clicks delete, this component calls this function with the transaction's ID, and the parent handles the actual deletion.

TODO 2 Accept Props in the Function Signature

📖 **What This Means**

Now that we defined the interface, we need to tell the function to actually receive those props. We use **destructuring** to pull out the individual values from the props object.

🔴 **You'll See This**

```
function TransactionItem() {
```

⬇️

🟢 **Change It To This**

```
function TransactionItem({ transaction, onDelete }: TransactionItemProps) {
```

Why: The `{ transaction, onDelete }` syntax destructures the props object so we can use `transaction` and `onDelete` directly instead of writing `props.transaction` and `props.onDelete` everywhere.

TODO 3 Render the Category Icon

📖 **What This Means**

Each category (Food, Transport, etc.) has an emoji and a CSS background color. The maps `categoryEmojis` and `categoryClasses` are already provided — we just need to look up the right values using the transaction's category.

🔴 **You'll See This**

```
{/* TODO 3: Render the category icon
    Use categoryEmojis[transaction.category] for the emoji
    Use categoryClasses[transaction.category] for the CSS class
    ... */}
```

⬇️

🟢 **Change It To This**

```
<div className={`category-icon ${categoryClasses[transaction.category]}`}>
  {categoryEmojis[transaction.category]}
</div>
```

Why: We use bracket notation (`categoryClasses[transaction.category]`) to look up the CSS class for this transaction's category. The template literal (`{`...`}`) lets us combine the base class `category-icon` with the dynamic category class like `food` or `transport`.

TODO 4 Display the Transaction Description

📖 **What This Means**

Replace the placeholder text with the actual transaction description from props.

🔴 **You'll See This**

```
<div className="transaction-description">Description here</div>
```

⬇️

🟢 **Change It To This**

```
<div className="transaction-description">
  {transaction.description}
</div>
```

Why: The curly braces `{}` let us embed a JavaScript expression inside JSX. `transaction.description` displays the actual description string from the transaction object.

TODO 5 Display the Category and Date

🔴 **You'll See This**

```
<div className="transaction-meta">
  <span>Category</span>
  <span>•</span>
  <span>Date</span>
</div>
```

⬇️

🟢 **Change It To This**

```
<div className="transaction-meta">
  <span>{transaction.category}</span>
  <span>•</span>
  <span>{transaction.date}</span>
</div>
```

Why: Same concept as TODO 4 — replace the static placeholder text with dynamic values from the transaction object.

TODO 6 Display the Amount with Formatting

📖 **What This Means**

Income should show in green with a "+" prefix. Expenses should show in red with a "-" prefix. We control this with a dynamic CSS class and a ternary operator for the sign.

🔴 **You'll See This**

```
<span className="transaction-amount">$0.00</span>
```

⬇️

🟢 **Change It To This**

```
<span className={`transaction-amount ${transaction.type}`}>
  {transaction.type === "expense" ? "-" : "+"}
  {formatCurrency(transaction.amount)}
</span>
```

Why: The CSS class `income` makes text green, `expense` makes it red. Since `transaction.type` is already either `"income"` or `"expense"`, we use it directly as the class name. The ternary operator (`condition ? ifTrue : ifFalse`) picks the right sign prefix.

TODO 7 Wire Up the Delete Button

📖 **What This Means**

When the user clicks the × button, we need to call the `onDelete` function with this transaction's ID. We use an arrow function so it only fires on click — not immediately on render.

🔴 **You'll See This**

```
<button className="delete-btn" title="Delete transaction">
  ×
</button>
```

⬇️

🟢 **Change It To This**

```
<button
  className="delete-btn"
  onClick={() => onDelete(transaction.id)}
  title="Delete transaction"
>
  ×
</button>
```

Why the arrow function? Writing `onClick={() => onDelete(transaction.id)}` creates a function that waits for a click. If we wrote `onClick={onDelete(transaction.id)}` instead, it would call `onDelete` immediately when the component renders — not what we want!

✅ **Checkpoint: TransactionItem Complete**

Save the file. You won't see this component on screen yet because nothing is rendering it — that comes later when we connect everything in App.tsx.

🧩 Part 4: SummaryCards Component

`src/components/SummaryCards.tsx`

This component displays three cards at the top of the app: Total Income, Total Expenses, and Balance.

TODO 1 Define the Props Interface

🔴 **You'll See This**

```
// TODO 1: Define a TypeScript interface called "SummaryCardsProps" with one property:
//         - summary: BudgetSummary
```

⬇️

🟢 **Change It To This**

```
interface SummaryCardsProps {
  summary: BudgetSummary;
}
```

Why: This component needs the calculated budget summary (income total, expense total, balance) to display in the three cards.

TODO 2 Accept Props in the Function Signature

🔴 You'll See This

```
function SummaryCards() {
```

⬇️

🟢 Change It To This

```
function SummaryCards({ summary }: SummaryCardsProps) {
```

TODO 3 Create the Three Summary Cards

📖 What This Means

Render three cards — one for income (green), one for expenses (red), and one for balance (blue, or orange if negative). Each card uses a different CSS class to control its color.

🔴 You'll See This

```
{/* TODO 3: Create three summary cards for Income, Expenses, and Balance.
    ... */}
```

⬇️

🟢 **Change It To This**

```
{/* Income Card */}
<div className="summary-card income">
  <div className="label">Total Income</div>
  <div className="amount">{formatCurrency(summary.totalIncome)}</div>
</div>

{/* Expenses Card */}
<div className="summary-card expenses">
  <div className="label">Total Expenses</div>
  <div className="amount">{formatCurrency(summary.totalExpenses)}</div>
</div>

{/* Balance Card */}
<div className={`summary-card balance ${summary.balance < 0 ? "negative" : ""}`}>
  <div className="label">Balance</div>
  <div className="amount">
    {summary.balance < 0 ? "-" : ""}
    {formatCurrency(summary.balance)}
  </div>
</div>
```

Why: Each card has a CSS class (`income`, `expenses`, `balance`) that controls the text color. For the balance card, we dynamically add the `negative` class when the balance is below zero — this changes the color to orange as a warning. The `formatCurrency` function (already provided) uses `Math.abs()`, so we add our own "-" prefix for negatives.

✅ **Checkpoint: SummaryCards Complete**

Save the file. This component is done — you'll see it on screen once we render it from App.tsx.

🧩 Part 5: TransactionForm Component

`src/components/TransactionForm.tsx`

This is the form that lets users add new transactions. It has an income/expense toggle, description input, amount input, category dropdown, and a submit button. This component has the most TODOs because forms require managing multiple pieces of state.

TODO 1 Define the Props Interface

🔴 You'll See This

```
// TODO 1: Define a TypeScript interface called "TransactionFormProps" with one proper
//         - onAddTransaction: a function that takes (description: string, amount: num
//           category: Category, type: "income" | "expense") and returns void
```

⬇️

🟢 **Change It To This**

```
interface TransactionFormProps {
  onAddTransaction: (
    description: string,
    amount: number,
    category: Category,
    type: "income" | "expense"
  ) => void;
}
```

Why: The parent component (App) will pass down a function that handles creating the transaction and adding it to the array. This form just collects user input and passes it up.

TODO 2 Accept Props in the Function Signature

🔴 **You'll See This**

```
function TransactionForm() {
```

⬇️

🟢 **Change It To This**

```
function TransactionForm({ onAddTransaction }: TransactionFormProps) {
```

TODO 3 Create State Variables for Form Fields

📖 What This Means

Each form input needs its own piece of state. When the user types, we update the state, and the input displays the current state value. This pattern is called a **controlled component** — React is the "single source of truth" for the input's value.

🔴 You'll See This

```
// TODO 3: Create state variables for the form fields:
//         - description (string, initially "")
//         - amount (string, initially "") — we use string because input values are st
//         - category (Category, initially "Food")
//         - type ("income" | "expense", initially "expense")
```

⬇️

🟢 Change It To This

```
const [description, setDescription] = useState("");
const [amount, setAmount] = useState("");
const [category, setCategory] = useState<Category>("Food");
const [type, setType] = useState<"income" | "expense">("expense");
```

Why is amount a string? HTML input elements always return strings. We store it as a string and convert to a number only when submitting. The TypeScript generics (`<Category>`, `<"income" | "expense">`) tell `useState` exactly what types the state can hold.

TODO 4 Write the Submit Handler

📖 What This Means

This function runs when the user clicks the submit button. It validates the input, calls the parent's callback function, and resets the form back to empty.

🔴 You'll See This

```
// TODO 4: Write a handleSubmit function that:
//         - Prevents the default form submission (e.preventDefault())
//         - Parses the amount string to a number using parseFloat()
//         - Validates: description must not be empty, amount must be a positive numbe
//         - Calls onAddTransaction with the form values
//         - Resets the form fields back to their initial values
```

⬇️

🟢 **Change It To This**

```
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();

  const parsedAmount = parseFloat(amount);

  // Validate inputs
  if (!description.trim()) return;
  if (isNaN(parsedAmount) || parsedAmount <= 0) return;

  onAddTransaction(description.trim(), parsedAmount, category, type);

  // Reset form
  setDescription("");
  setAmount("");
  setCategory("Food");
};
```

Key concepts:

- `e.preventDefault()` — stops the browser from reloading the page (default form behavior)
- `.trim()` — removes whitespace from both ends of the string
- `parseFloat()` — converts a string to a decimal number
- `isNaN()` — checks if the value is "Not a Number"
- We `return` early if validation fails — the form won't submit with bad data

---

TODO 5 Add the onSubmit Handler to the Form

🔴 You'll See This

```
<form /* TODO 5: Add onSubmit handler */>
```

⬇️

🟢 Change It To This

```
<form onSubmit={handleSubmit}>
```

TODO 6–7 Wire Up the Expense Toggle Button

📖 What This Means

The toggle buttons need two things: a dynamic CSS class to show which is active, and an onClick handler to switch the type.

🔴 You'll See This

```
<button
  type="button"
  className="type-btn"
  // TODO 6: Add the class "active-expense" when type === "expense"
  // TODO 7: Add onClick to set type to "expense" and category to "Food"
>
  Expense
</button>
```

⬇️

🟢 Change It To This

```
<button
  type="button"
  className={`type-btn ${type === "expense" ? "active-expense" : ""}`}
  onClick={() => {
    setType("expense");
    setCategory("Food");
  }}
>
  Expense
</button>
```

Why `type="button"`**?** Without it, buttons inside a `<form>` default to `type="submit"`, which would trigger form submission on every click.

TODO 8–9 Wire Up the Income Toggle Button

🔴 You'll See This

```
<button
  type="button"
  className="type-btn"
  // TODO 8: Add the class "active-income" when type === "income"
  // TODO 9: Add onClick to set type to "income" and category to "Income"
>
  Income
</button>
```

⬇️

🟢 Change It To This

```
<button
  type="button"
  className={`type-btn ${type === "income" ? "active-income" : ""}`}
  onClick={() => {
    setType("income");
    setCategory("Income");
  }}
>
  Income
</button>
```

TODO 10 Connect the Description Input

📖 What This Means

A **controlled input** has two parts: `value` (what it displays) and `onChange` (how it updates). React becomes the single source of truth — the input always shows what's in state.

🔴 You'll See This

```
<input
  id="description"
  type="text"
  placeholder="e.g., Grocery shopping"
  // TODO 10: Set value to description state and add onChange to update it
/>
```

⬇️

🟢 Change It To This

```
<input
  id="description"
  type="text"
  placeholder="e.g., Grocery shopping"
  value={description}
  onChange={(e) => setDescription(e.target.value)}
/>
```

Why: `e.target.value` is the current text in the input. On every keystroke, `onChange` fires, updates state, and React re-renders with the new value.

TODO 11 Connect the Amount Input

🔴 You'll See This

```
<input
  id="amount"
  type="number"
  placeholder="0.00"
  min="0.01"
  step="0.01"
  // TODO 11: Set value to amount state and add onChange to update it
/>
```

⬇️

🟢 Change It To This

```
<input
  id="amount"
  type="number"
  placeholder="0.00"
  min="0.01"
  step="0.01"
  value={amount}
  onChange={(e) => setAmount(e.target.value)}
/>
```

TODO 12–14 Conditional Category Dropdown

📖 What This Means

The category dropdown should only appear for expenses — not income. This is **conditional rendering**. The syntax `{condition && (jsx)}` means "only render this when the condition is true."

🔴 You'll See This

```
{/* TODO 12: Only render this div when type === "expense" */}
<div className="form-group">
  <label htmlFor="category">Category</label>
  <select
    id="category"
    // TODO 13: Set value to category state and add onChange to update it
  >
    {/* TODO 14: Map over expenseCategories to render option elements */}
  </select>
</div>
```

⬇️

🟢 **Replace the Entire Block Above With This**

```
{type === "expense" && (
  <div className="form-group">
    <label htmlFor="category">Category</label>
    <select
      id="category"
      value={category}
      onChange={(e) => setCategory(e.target.value as Category)}
    >
      {expenseCategories.map((cat) => (
        <option key={cat} value={cat}>
          {cat}
        </option>
      ))}
    </select>
  </div>
)}
```

Key concepts:

- `{type === "expense" && (...)}` — only renders the dropdown when type is "expense"
- `as Category` — TypeScript type assertion (the select returns a generic string, but we know it's a Category)
- `.map()` — transforms the categories array into `<option>` elements
- `key={cat}` — React requires a unique key on each item in a list

TODO 15 Dynamic Submit Button Text

🔴 You'll See This

```
<button type="submit" className="submit-btn">
  {/* TODO 15: Show "Add Income" or "Add Expense" based on the type state */}
  Add Transaction
</button>
```

⬇️

🟢 Change It To This

```
<button type="submit" className="submit-btn">
  Add {type === "income" ? "Income" : "Expense"}
</button>
```

✅ Checkpoint: TransactionForm Complete

Save the file. The form will render on screen once you hook it up in App.tsx.

🧩 Part 6: TransactionList Component

`src/components/TransactionList.tsx`

This component displays the list of transactions with category filter buttons at the top. It receives the full list from the parent and manages the active filter locally.

TODO 1 Define the Props Interface

🔴 **You'll See This**

```
// TODO 1: Define a TypeScript interface called "TransactionListProps" with:
//         - transactions: Transaction[]
//         - onDeleteTransaction: a function that takes (id: number) and returns void
```

⬇️

🟢 **Change It To This**

```
interface TransactionListProps {
  transactions: Transaction[];
  onDeleteTransaction: (id: number) => void;
}
```

TODO 2 Accept Props in the Function Signature

🔴 You'll See This

```
function TransactionList() {
```

⬇️

🟢 Change It To This

```
function TransactionList({ transactions, onDeleteTransaction }: TransactionListProps)
```

TODO 3 Create Filter State

🔴 You'll See This

```
// TODO 3: Create a state variable called "activeFilter" of type ("All" | Category),
//         initialized to "All"
```

⬇️

🟢 Change It To This

```
const [activeFilter, setActiveFilter] = useState<"All" | Category>("All");
```

TODO 4 Filter the Transactions

📖 What This Means

Based on the selected filter, show either all transactions or only those matching a specific category. This uses a ternary expression to create a new (filtered) array without changing the original.

🔴 You'll See This

```
// TODO 4: Create a "filteredTransactions" variable that:
//         - If activeFilter is "All", returns all transactions
//         - Otherwise, returns only transactions matching the selected category
//         Hint: Use a ternary with .filter()
```

⬇️

🟢 Change It To This

```
const filteredTransactions =
  activeFilter === "All"
    ? transactions
    : transactions.filter((t) => t.category === activeFilter);
```

Why: If "All" is selected, we use the full array. Otherwise, `.filter()` returns a new array with only transactions whose category matches the active filter.

TODO 5 Show the Transaction Count

🔴 You'll See This

```
{/* TODO 5: Show the count of filtered transactions ... */}
```

⬇️

🟢 Change It To This

```
<span className="transaction-count">
  {filteredTransactions.length}{" "}
  {filteredTransactions.length === 1 ? "item" : "items"}
</span>
```

TODO 6 Render the Filter Buttons

📖 What This Means

Map over the `categoryFilters` array to create a row of buttons. The active button gets a highlighted style. Clicking updates the filter state, which re-renders the filtered list.

🔴 You'll See This

```
{/* TODO 6: Map over categoryFilters to create filter buttons. ... */}
```

⬇️

🟢 **Change It To This**

```
{categoryFilters.map((filter) => (
  <button
    key={filter}
    className={`filter-btn ${activeFilter === filter ? "active" : ""}`}
    onClick={() => setActiveFilter(filter)}
  >
    {filter}
  </button>
))}
```

TODO 7 Render Transactions or Empty State

📖 **What This Means**

If there are no transactions (or none match the filter), show a friendly empty message. Otherwise, render a `TransactionItem` for each transaction. This uses a ternary for **conditional rendering**.

🔴 **You'll See This**

```
{/* TODO 7: Conditionally render either:
    A) An empty state when filteredTransactions has no items ...
    B) A list of TransactionItem components ... */}
```

⬇️

🟢 **Change It To This**

```
{filteredTransactions.length === 0 ? (
  <div className="empty-state">
    <div className="empty-icon">💰</div>
    <p>No transactions yet</p>
    <p className="hint">Add your first transaction to get started</p>
  </div>
) : (
  filteredTransactions.map((transaction) => (
    <TransactionItem
      key={transaction.id}
      transaction={transaction}
      onDelete={onDeleteTransaction}
    />
  ))
)}
```

Key concepts:

- `condition ? (if true) : (if false)` — ternary for conditional rendering
- `key={transaction.id}` — each list item needs a unique key for React to efficiently update the DOM
- We pass `onDeleteTransaction` to each TransactionItem so the delete button works

✅ **Checkpoint: TransactionList Complete**

Save the file. Now for the final piece — connecting everything in App.tsx!

🧩 Part 7: App Component (Bringing It All Together)

`src/App.tsx`

The App component is the "brain" of your application. It owns the transaction data, provides functions to add and delete, calculates the summary, and passes everything to child components as props.

TODO 1 Create the State

📖 What This Means

The App needs to store the array of all transactions and a counter for unique IDs. This is the "source of truth" — all other components receive data from here through props.

🔴 You'll See This

```
// TODO 1: Create a state variable called "transactions" to hold an array of Transacti
//         Initialize it with an empty array.
//         Also create a state variable called "nextId" starting at 1 (for generating
```

⬇️

🟢 Change It To This

```
const [transactions, setTransactions] = useState<Transaction[]>([]);
const [nextId, setNextId] = useState(1);
```

Why: `useState<Transaction[]>([])` creates state holding an array of Transaction objects, starting empty. `nextId` is a counter — each new transaction gets this number as its ID, then we increment it.

TODO 2 Write the calculateSummary Function

📖 What This Means

This function takes the transactions array and calculates three numbers: total income, total expenses, and balance. It uses `.filter()` to separate income from expenses, and `.reduce()` to add up the amounts.

🔴 You'll See This

```
// TODO 2: Write a function called "calculateSummary" that takes the transactions arra
//         and returns a BudgetSummary object with:
//         - totalIncome: sum of all transactions where type === "income"
//         - totalExpenses: sum of all transactions where type === "expense"
//         - balance: totalIncome - totalExpenses
//         Hint: Use .filter() and .reduce()
```

⬇️

🟢 **Change It To This**

```
const calculateSummary = (txns: Transaction[]): BudgetSummary => {
  const totalIncome = txns
    .filter((t) => t.type === "income")
    .reduce((sum, t) => sum + t.amount, 0);

  const totalExpenses = txns
    .filter((t) => t.type === "expense")
    .reduce((sum, t) => sum + t.amount, 0);

  return {
    totalIncome,
    totalExpenses,
    balance: totalIncome - totalExpenses,
  };
};
```

How .reduce() works: It starts with an initial value (0) and "accumulates" a result by running a function on each item. `(sum, t) => sum + t.amount` means "take the running total and add this transaction's amount." After every item, you get the grand total.

TODO 3 Write handleAddTransaction

📖 **What This Means**

This function creates a new transaction object and adds it to the beginning of the array. The key React principle: **never mutate state directly**. Instead of pushing to the array, we create a new array with the spread operator.

🔴 You'll See This

```
// TODO 3: Write a function called "handleAddTransaction" that:
//         - Takes description (string), amount (number), category (Category), type ("
//         - Creates a new Transaction object with a unique id, today's date, and the
//         - Adds it to the beginning of the transactions array using setTransactions
//         - Increments nextId
//         Hint: For today's date use: new Date().toISOString().split("T")[0]
```

⬇️

🟢 Change It To This

```
const handleAddTransaction = (
  description: string,
  amount: number,
  category: Category,
  type: "income" | "expense"
) => {
  const newTransaction: Transaction = {
    id: nextId,
    description,
    amount,
    category,
    type,
    date: new Date().toISOString().split("T")[0],
  };

  setTransactions([newTransaction, ...transactions]);
  setNextId(nextId + 1);
};
```

Key concepts:

- We create a new `Transaction` object matching the TypeScript interface
- `new Date().toISOString().split("T")[0]` gives today's date as "2026-02-24"
- `[newTransaction, ...transactions]` — the spread operator creates a new array with the new item first. Never mutate state directly in React!
- `setNextId(nextId + 1)` — increment the counter for the next transaction

TODO 4 Write handleDeleteTransaction

📖 What This Means

To "delete" a transaction in React, we don't remove it from the array — we create a new array that contains everything except the one we want to remove.

🔴 You'll See This

```
// TODO 4: Write a function called "handleDeleteTransaction" that:
//         - Takes an id (number)
//         - Removes the transaction with that id from the array
//         Hint: Use .filter()
```

⬇️

🟢 Change It To This

```
const handleDeleteTransaction = (id: number) => {
  setTransactions(transactions.filter((t) => t.id !== id));
};
```

Why: `.filter((t) => t.id !== id)` returns a new array containing every transaction whose ID does not match the one we're deleting. The original array is untouched — React gets a brand new array and re-renders.

TODO 5 Calculate the Summary

🔴 **You'll See This**

```
// TODO 5: Call calculateSummary to get the current summary
```

⬇️

🟢 **Change It To This**

```
const summary = calculateSummary(transactions);
```

Why: This runs on every render, so the summary cards always reflect the current transactions. When a transaction is added or deleted, React re-renders, this line re-runs, and the cards update.

TODO 6–8 Render the Child Components

📖 **What This Means**

This is where everything comes together. You render your three completed components and pass them the data and functions they need through props. This is React's core pattern: **data flows down** from parent to children.

🔴 You'll See This

```
{/* TODO 6: Render the SummaryCards component and pass the summary as a prop */}

<div className="main-content">
  {/* TODO 7: Render the TransactionForm component and pass handleAddTransaction as a

  {/* TODO 8: Render the TransactionList component and pass transactions and handleDel
</div>
```

⬇️

🟢 Change It To This

```
<SummaryCards summary={summary} />

<div className="main-content">
  <TransactionForm onAddTransaction={handleAddTransaction} />

  <TransactionList
    transactions={transactions}
    onDeleteTransaction={handleDeleteTransaction}
  />
</div>
```

Why: Each component receives exactly the props defined in its interface. `SummaryCards` gets the summary object. `TransactionForm` gets the function to call when adding. `TransactionList` gets the data to display and the function to call when deleting.

🎉 Your App Should Be Working!

Save the file and check the browser. You should see:

- Three summary cards (all showing $0.00)
- The transaction form on the left
- The empty state message on the right ("No transactions yet")

Try adding a transaction — fill in the form and click "Add Expense". The transaction should appear in the list and the summary cards should update automatically!

🚀 Part 8: Deploy to GitHub and Vercel

Once your app works locally, push it to GitHub and deploy to Vercel so it's live on the web.

Push to GitHub

Since you cloned the starter repo, the git remote currently points to my repository. You need to create your own repo and push there.

1 Create a GitHub Repository

Go to github.com/new and create a new **public** repository. Name it something like `student-budget-tracker`. Do **not** initialize it with a README.

2 Update the Remote and Push

```
# Remove the original remote (the starter repo)
git remote remove origin

# Add your own repo as the remote
git remote add origin https://github.com/YOUR-USERNAME/student-budget-tracker.git

# Commit your work and push
git add -A
git commit -m "Student Budget Tracker - IT 431 Project"
git push -u origin main
```

> ⚠️ **Don't Push node_modules**
>
> The `.gitignore` file is already set up to exclude `node_modules/`. Never push `node_modules` to GitHub — it contains thousands of files that anyone can reinstall with `npm install`.

Deploy to Vercel

3 Create a Vercel Account

Go to vercel.com and sign up with your GitHub account (free tier is all you need).

4 Import Your Repository

1. Click **"Add New..."** → **"Project"**
2. Select your `student-budget-tracker` repository from the list
3. Vercel should auto-detect it as a Vite project
4. Click **"Deploy"**
5. Wait about 60 seconds for the build to finish

5 Verify Your Deployment

Vercel gives you a URL like `https://student-budget-tracker-abc123.vercel.app`. Open it and make sure your app works.

> 💡 **Automatic Deployments**
>
> After connecting your repo, Vercel automatically redeploys every time you push to `main`. Fix a bug and push — the live site updates automatically!

📦 Submission Requirements

Submit the following on Canvas:

1. **GitHub Repository URL** — your public repo (e.g., `https://github.com/yourname/student-budget-tracker`)
2. **Vercel Live URL** — your deployed app (e.g., `https://student-budget-tracker-xyz.vercel.app`)

Grading Rubric (200 Points)

Criteria	Points	What I'm Looking For
TypeScript Interfaces & Type Safety	30	Props interfaces defined for all 4 components. TypeScript types used correctly throughout (no `any`). Generics on `useState` where appropriate.
Component Structure & Props	40	All components accept and use props correctly. Data flows from App → child components. Components are properly imported and rendered.
State Management (useState)	40	State variables created correctly for transactions, form fields, and filter. State updates create new arrays/values (no direct mutation). Form resets after submission.
Event Handling	30	Form submission with `preventDefault` and validation. Delete buttons call the correct handler. Toggle buttons switch income/expense. Controlled inputs with `value` + `onChange`.
List Rendering & Conditional Display	30	Lists use `.map()` with unique keys. Filter logic works correctly. Empty state shown when no transactions. Category dropdown conditionally rendered.
GitHub & Vercel Deployment	20	Code pushed to a public GitHub repository. App deployed and accessible on Vercel. Both URLs submitted.
Code Quality	10	Clean, readable code. Proper indentation. TODO comments removed or replaced with meaningful comments. No console errors in the browser.
Total	**200**	

🔧 Common Issues & Troubleshooting

Problem	Solution
Red errors about "type-only import"	Make sure imports use `import type` for types: `import type { Transaction } from "../types"`
Page is blank / nothing renders	Check the browser console (F12) for errors. Usually a typo in a component name or missing import.
Form submits but page reloads	Make sure you have `e.preventDefault()` in your handleSubmit function.
Transactions don't appear in the list	Check that you're passing `transactions` (not `filteredTransactions`) from App to TransactionList.
Delete button doesn't work	Use an arrow function: `onClick={() => onDelete(id)}`, not `onClick={onDelete(id)}`
Vercel build fails	Run `npm run build` locally first. Fix any TypeScript errors before pushing.
Styling looks wrong	CSS class names are case-sensitive. Check they match exactly. Make sure `App.css` is imported in App.tsx.

💡 General Debugging Tips

- **Read the error message.** TypeScript errors usually tell you exactly what's wrong and where.
- **Check the browser console** (F12 → Console tab) for runtime errors.
- **Add** `console.log()` to check if functions are running and what values they receive.
- **Work one TODO at a time.** Save and check the browser after each change.
- **Compare with types.ts** — the interfaces define the exact shape your data needs.